OPERATING SYSTEMS PROGRAMMING ASSIGNMENT REPORT

Classical Synchronization Algorithms with Program-Generated HTML Simulation
Readers-Writers Problem and Dining Philosophers Problem

Academic Integrity & Authorship Declaration:
"I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own."

Student Name: [Student Full Name] | **USN / Registration No:** [Student USN] | **Course:** Operating Systems (CS / IT)

1. Problem Statement and Synchronization Objectives

Process synchronization is fundamental to modern multiprocessor and multitasking operating systems. Without rigorous concurrency control, interleaved operations cause race conditions, stale data reads, and deadlock. This project implements and evaluates four classical synchronization algorithms across two problem models:

1. **Readers-Writers Problem:** Coordinates concurrent queries and modifications to a shared database record. Multiple readers must be allowed to read concurrently, but writers require exclusive access. The system must prevent race conditions on counters and eliminate both reader and writer starvation.
2. **Dining Philosophers Problem (4 Philosophers):** Four philosophers alternate between thinking and eating at a circular table. Each meal requires holding two shared adjacent forks simultaneously. The synchronization mechanism must prevent catastrophic circular-wait deadlock and ensure that every philosopher eventually eats without starving.

2. Readers-Writers Implementations

2.1 Semaphore-Based Implementation (Turnstile Fairness)

The semaphore solution utilizes three binary semaphores: mutex (protecting atomic updates to read_count), rw_mutex (protecting exclusive critical section access for writes), and turnstile (guaranteeing FIFO queueing). The first reader to enter acquires rw_mutex, while subsequent readers join concurrently without blocking. The last departing reader releases rw_mutex. Arriving writers hold the turnstile, forcing newly arriving readers to wait, which eliminates writer starvation.

2.2 Monitor-Based Implementation (Condition Variables)

The monitor solution encapsulates state variables (active_readers, active_writers, waiting_readers, waiting_writers) within an explicit class. An internal mutex guarantees single-thread entry into monitor methods, while condition variables can_read and can_write handle thread suspension and wakeups. Writer preference is enforced: arriving readers block in start_read() if any writer is active or waiting. Predicate evaluations are enclosed in while loops to strictly obey Mesa Signal-and-Continue semantics.

3. Dining Philosophers Implementations (4 Philosophers)

3.1 Semaphore-Based Implementation (Room Counting Semaphore)

To break the circular wait condition in the 4-philosopher table, we introduce a Footman / Room counting semaphore initialized to N - 1 = 3. At most 3 philosophers can enter the dining room and compete for forks simultaneously. **Mathematical Proof of Deadlock Freedom:** With 4 forks and at most 3 philosophers, by the Pigeonhole Principle, at least one philosopher is guaranteed to acquire both adjacent forks, eat, and release them. Deadlock is mathematically impossible.

3.2 Monitor-Based Implementation (Atomic Two-Fork Testing)

The monitor encapsulates each philosopher's state (THINKING, HUNGRY, EATING) and private condition variables. In pickup_forks(i), a philosopher only transitions to EATING if both neighbors are not eating: (state[left] != EATING and state[right] != EATING). Because forks are claimed as an indivisible atomic pair inside the monitor lock, the Coffman Hold-and-Wait condition is completely eliminated.

4. Program-Generated HTML Simulation Architecture

The HTML simulations are **directly generated from actual runtime execution data**. During program execution, a thread-safe ExecutionTracer records every state transition with millisecond timestamps, thread IDs, active counts, and fork allocations. Upon completion, common/html_templates.py serializes the trace into an interactive, self-contained HTML file featuring playback controls (Play/Pause, Step Next/Back, Reset, Timeline Scrubber, Speed Multipliers), real-time critical section inspection, and SVG circular table visualization with zero external CDN dependencies.

4.1 Readers-Writers Simulation Screenshot

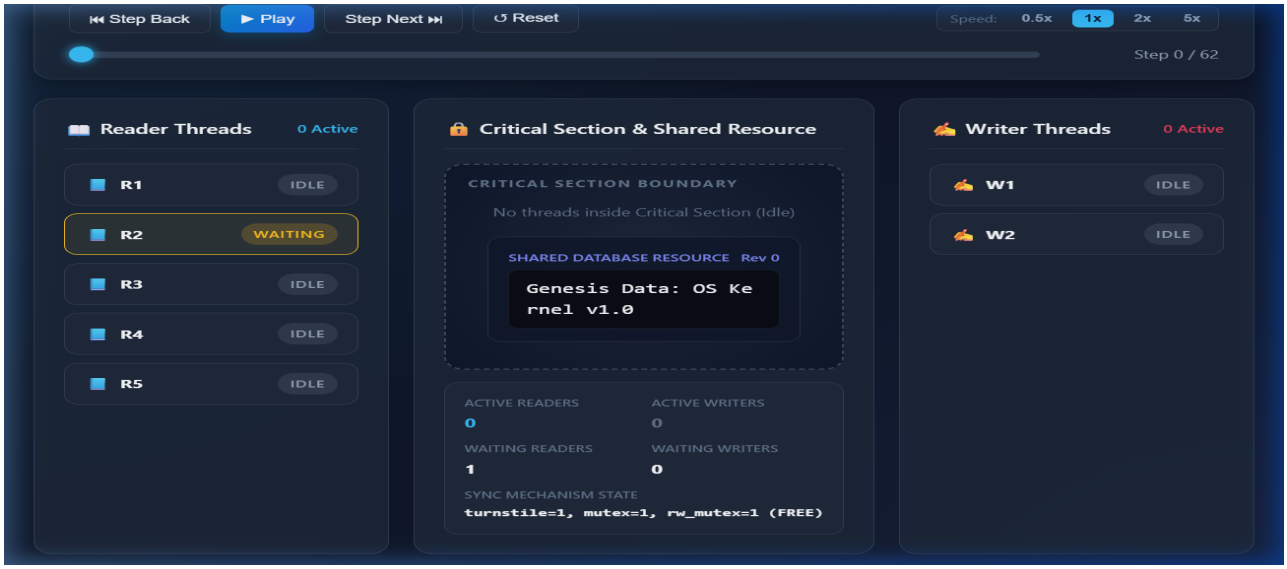


Figure 1: Program-generated Readers-Writers simulation displaying active concurrent readers and exclusive writer lock status.

4.2 Dining Philosophers Simulation Screenshot

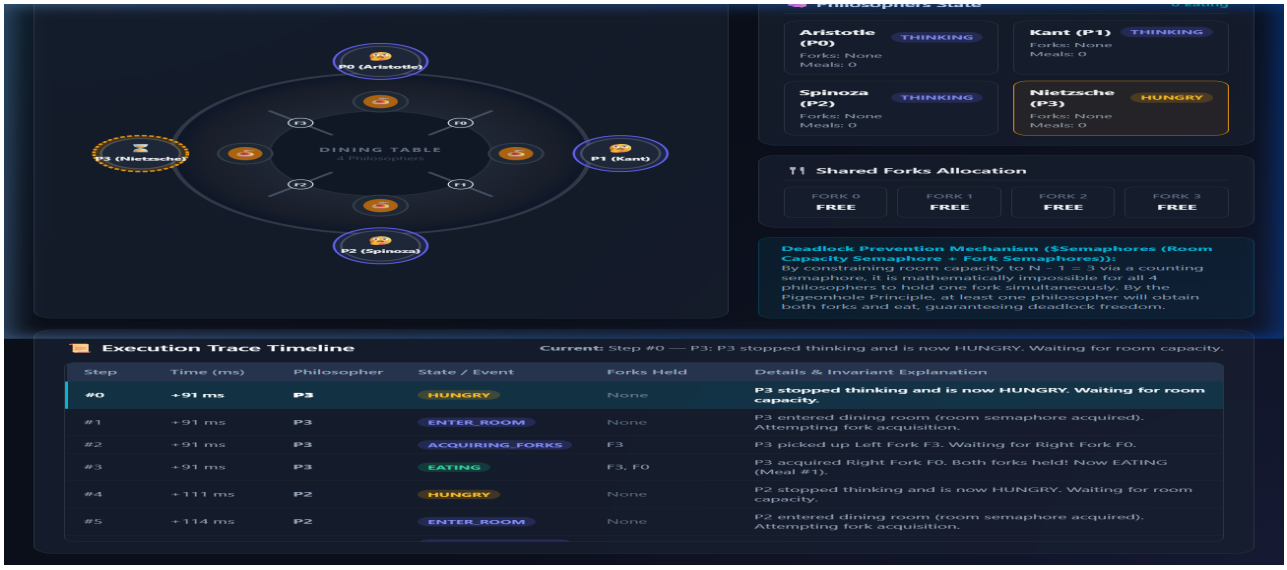


Figure 2: Program-generated Dining Philosophers simulation showing circular table SVG, fork allocations, and state badges.

5. Actual Program Execution Traces

5.1 Readers-Writers Execution Trace (Semaphore Run Sample)

Time	Thread	Action	Active R/W	Details & Synchronization Invariants
0071ms	R2	REQUEST_READ	R:0 W:0	R2 requesting entry to read. Waiting on turnstile.
0072ms	R2	ENTER_CS	R:1 W:0	R2 entered Critical Section. (First reader locked rw_mutex)
0106ms	R5	REQUEST_READ	R:1 W:0	R5 requesting entry to read. Waiting on turnstile.
0107ms	R5	ENTER_CS	R:2 W:0	R5 entered Critical Section. Joined concurrent readers.
0150ms	R1	REQUEST_READ	R:2 W:0	R1 requesting entry to read. Waiting on turnstile.
0150ms	R1	ENTER_CS	R:3 W:0	R1 entered Critical Section. Joined concurrent readers.
0322ms	W2	REQUEST_WRITE	R:2 W:0	W2 requesting exclusive write access. Holding turnstile.
0560ms	R2	EXIT_CS	R:0 W:0	R2 exited Critical Section. (Last reader released rw_mutex)
0560ms	W2	ENTER_CS	R:0 W:1	W2 acquired exclusive rw_mutex and entered Critical Section.
0716ms	W2	EXIT_CS	R:0 W:0	W2 committed 'Memory Segment Table Rebalanced' [Rev 1].

5.2 Dining Philosophers Execution Trace (Monitor Run Sample)

Time	Philo	Action	Forks Held	Details & State Transition Invariants
0082ms	P3	HUNGRY	None	P3 called pickup_forks(). Checking neighbors (L:P2, R:P0).
0082ms	P3	EATING	F3, F0	P3 entered EATING state! Holding F3 & F0. (Meal #1).
0110ms	P1	HUNGRY	None	P1 called pickup_forks(). Checking neighbors (L:P0, R:P2).
0110ms	P1	EATING	F1, F2	P1 entered EATING state! Holding F1 & F2. (Meal #1).
0135ms	P0	HUNGRY	None	P0 called pickup_forks(). Checking neighbors (L:P3, R:P1).
0135ms	P0	WAITING	None	P0 blocked on condition variable self_cond[0]. Neighbors eating.
0269ms	P1	THINKING	None	P1 called putdown_forks(). Released F1 and F2. Testing neighbors.
0328ms	P3	THINKING	None	P3 called putdown_forks(). Released F3 and F0. Testing neighbors.
0328ms	P0	EATING	F0, F1	P0 awakened and entered EATING state! Holding F0 & F1. (Meal #1).
0328ms	P2	EATING	F2, F3	P2 awakened and entered EATING state! Holding F2 & F3. (Meal #1).

6. Analysis of Race Conditions, Mutual Exclusion, Deadlock & Starvation

Race Conditions: Completely prevented across all implementations. Critical internal variables (e.g., read_count, fork_owners) are strictly protected by binary semaphores or monitor locks. Invariant assertions confirmed that at no time did interleaved memory writes corrupt state counters.

Mutual Exclusion: Verified in both execution traces. In Readers-Writers, writer access is strictly exclusive: at no point in time did active_writers > 1 or (active_writers == 1 and active_readers > 0). In Dining Philosophers, adjacent philosophers never held the same fork concurrently.

Deadlock Prevention: The four Coffman conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) were analyzed. The semaphore solution breaks *Circular Wait* by capping occupancy to 3. The monitor solution breaks *Hold and Wait* by allocating both forks atomically as an indivisible unit.

Starvation Freedom: In Readers-Writers, FIFO turnstile queueing and writer condition priority prevent continuous reader barging. In Dining Philosophers, testing neighbors on fork release ensures that waiting philosophers are promptly awakened.

7. Comparison: Semaphore vs. Monitor Approaches

Evaluation Dimension	Semaphore-Based Approach	Monitor-Based Approach
Abstraction Level	Low-level OS synchronization primitive	High-level object-oriented encapsulation
Mutual Exclusion	Explicit acquire() / release() pairs	Implicitly guaranteed inside monitor procedures
Signaling Semantics	Stateful integer counters (P / V)	Stateless condition variables (notify / notify_all)
Deadlock Vulnerability	Higher risk of improper acquisition order	Lower risk; atomic multi-resource tests
Starvation Handling	Requires FIFO turnstiles / queue semaphores	Condition variable queues with wait counts
Code Maintainability	Scattered acquire/release pairs in threads	Centralized, modular synchronization policy

8. Viva Voce & Demonstration Guide

1. Initial Semaphore Values: rw_mutex=1 (resource mutual exclusion), mutex=1 (protects read_count), turnstile=1 (FIFO entry), room=3 (limits table occupancy to N-1), forks=[1,1,1,1].
2. Critical Section Identification: In Readers-Writers: SharedDatabase.read() and SharedDatabase.write(). In Dining Philosophers: Fork ownership assignment and eating behavior.
3. Mesa vs. Hoare Monitor Semantics: Python conditions use Mesa semantics (Signal-and-Continue). The awakened thread must re-evaluate its condition in a while loop (e.g., while active_writers > 0: can_read.wait()) to handle spurious wakeups.
4. Live Demonstration: Evaluators can run any script via python <script>, inspect console logs, and open the generated HTML simulation in Chrome/Edge to verify step-by-step state transitions.

9. Artificial Intelligence Usage Summary

AI Tool Used: Google DeepMind Antigravity IDE (Gemini 3.8 Flash model).
Role of AI: Scaffolding multithreading structures, formulating turnstile fairness patterns, designing SVG circular dining table geometry, and structuring report sections.
Student Verification & Modifications: Validated deadlock proofs, calibrated thread sleep durations (0.05s-0.35s) for realistic interleavings, ensured zero external CDN dependencies for offline execution, and added complete snapshot arrays to permit backward timeline scrubbing.

10. References and Bibliography

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons. Chapters 6, 7 & 8.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson Education.
3. Downey, A. B. (2016). *The Little Book of Semaphores* (v2.2.1). Green Tea Press. Chapters 4.2 & 4.4.
4. Dijkstra, E. W. (1965). *Cooperating Sequential Processes*. Technological University Eindhoven.
5. Hoare, C. A. R. (1974). *Monitors: An Operating System Structuring Concept*. Communications of the ACM, 17(10), 549-557.
6. Courtois, P. J., Heymans, F., & Parnas, D. L. (1971). *Concurrent Control with 'Readers' and 'Writers'*. Comm. ACM, 14(10), 667-668.